# LLM Access Setup — LangChain Chat Completion Notebook

These steps get you from zero to a working LLM call in the `langchain_chat_completion.ipynb` notebook. This uses Intuit's internal LXS (LLM execution service), **not** an OpenAI API key.

> **Environment note:** Use the **E2E** environment, not prod. You'll need the E2E versions of both the Identity endpoint and the LXS endpoint.

---

## What you will navigate to get the following in DevPortal:

From the TL Agentic AI Training project:
https://devportal.intuit.com/app/dp/project/5699322921774834754/assets

- **App secret** - one shared secret for the whole cohort/class at: https://devportal.intuit.com/app/dp/resource/4258171041006342954/credentials/secrets
- **Experience ID** - one shared ID for the whole cohort at: https://devportal.intuit.com/app/dp/resource/7716935554827798552/overview

Do not share these outside the class. The Tech Learning team will rotate the secret after the cohort ends.

---

## Step 1: Generate an Access Token

Run this in a terminal or an API client (Postman, Intuit API client). Replace `<replace_with_app_secret>` with the secret shared in TinCan.

```
curl --location 'https://identityinternal.api.intuit.com/signin/graphql' \
--header 'Authorization: Intuit_IAM_Authentication
intuit_appid="Intuit.aifabric.genos.agenticaitrainingclient",intuit_app_secret="<repla
ce_with_app_secret>"' \
--header 'Content-Type: application/json' \
--data-raw '{"query":"mutation identityTestSignInWithPassword($input:
Identity_TestSignInWithPasswordInput!) {\n identityTestSignInWithPassword(input:
$input) {\n accessToken\n
legacyAuthId\n}\n}","variables":{"input":{"username":"iamtestpass_616829375333307","pa
ssword":"Intuit01-
","tenantId":"50000003","intent":{"appGroup":"QBO","assetAlias":"Intuit.aifabric.genos
.agenticaitrainingclient"}}}}'
```

- Confirm you're hitting the **identity E2E** endpoint (as shown above), not the prod identity URL.
- The response returns an `accessToken` — copy this; you'll need it in Step 2.

## Step 2: Build the PrivateAuth+ headers

In the notebook, find the `AUTHN_STRING` / `INTUIT_AUTHN_HEADERS` cell and fill in three placeholders:

| Placeholder | Value |
|---|---|
| `intuit_app_secret` | Same app secret from TinCan (Step 1) |
| `intuit_token` | The `accessToken` you got back from Step 1 |
| `intuit_experience_id` (in `INTUIT_AUTHN_HEADERS`) | Experience ID shared in TinCan — this is currently hard-coded in the notebook to the instructor's own ID, so replace it with the cohort's shared ID |

Leave `intuit_appid` (`Intuit.aifabric.genos.agenticaitrainingclient`) and `intuit_userid` as-is unless your instructor tells you otherwise.

---

## Step 3: Install dependencies

Run the first notebook cell:

```
%pip install --quiet -U langchain_openai langchain_core
%pip install requests
```

---

## Step 4: Run the chat completion cell

The `ChatOpenAI` client is pre-configured to route through LXS rather than OpenAI directly:

- `api_key="anything"` — not used, since auth happens via the Intuit headers
- `base_url="https://llmexecution.api.intuit.com/v3/gpt-4o-2024-05-13/"` — **confirm this is the E2E LXS endpoint** with your instructor; the model ID is embedded in the path, not the `model` field
- `extra_headers={**INTUIT_AUTHN_HEADERS}` — this is what actually authenticates your call

Run the cell with the sample messages. If everything is wired correctly, you'll see a streamed/completed response printed below the cell (as in the example: the marginal tax rate walkthrough).

---

## Troubleshooting

| Symptom | Likely cause |
|---|---|
| Auth error on the identity curl | Wrong app secret, or hitting prod instead of E2E identity URL |
| Auth error calling LXS | Token expired (re-run Step 1), wrong/missing experience ID, or hitting prod LXS instead of E2E |
| `extra_headers` warning in output | Expected/benign — LangChain just flags that `extra_headers` isn't a standard param; it still passes through correctly |

---

## If you want to go further (optional)

- **Building your own MCP server:** if the course also has you stand up an MCP server, do this **locally with Docker/Rancher/Podman** rather than deploying to AWS/E2E — it's faster to iterate and you'll still learn the protocol fundamentals. The docs for the Services Paved Road are at: https://devportal.intuit.com/app/dp/capability/CAP-2297/capabilityDocs/main/docs/mcp/mcp.md under "Building an MCP Server" if you plan to eventually publish one.

- **Own OpenAI-client notebooks:** if you'd rather not use the LangGraph/LangChain pattern, equivalent plain-OpenAI-client notebooks exist in AI Workbench and follow the same auth pattern.